

Name	Page
 db Medications and Lab Tests from XML.....Database Message Extraction	105



no-op eICR Msgs To Parsing Route

Filter

No-operation

On ECLIR - eCR Condition Checks route

Notes

Purpose: This No-operation filter sends the output from the *db Lookup Condition Code ID from XML* Database Lookup filter to the *ECLIR - eICR Message Parsing* route.

Destinations

Name	Page
 no-op eICR Msgs In.....No-operation	152



db Medications and Lab Tests from XML

Filter

Database Message Extraction

On ECLIR - eCR Condition Checks route

Notes

Purpose:

Both queries in this Database Message Extraction filter iterate through each `conditionCode` element created in the XML output from the *js Build XML for db query* JavaScript filter.

Each query returns all lab tests relevant to each condition from the `LABORATORY_TEST_CODES` table and all medications relevant to each condition from the `MEDICATION_CODES` table.

The output is an XML message that is sent to the *ECLIR - eICR Message Parsing* route to be used for logic in the *js Parse eICR to XML for DB Inserts* JavaScript filter to find any relevant lab tests or medications contained in the eICR document.

Configuration Properties

Maximum Concurrency	1
Database	[REDACTED]
Host	[REDACTED]
Port	[REDACTED]
Database Name	[REDACTED]
Username	[REDACTED]
Password	[REDACTED]

Configuration Properties		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File: Database Message Extraction.xml		
<pre><statement name="MEDICATION_CODES" ide:type="executeSql" iterate="conditionCode" ide:summary="Return all medications relevant to the condition from the MEDICATION_CODES table." ide:allowCompoundStatements="true"> <sql>SELECT MEDICATION_CODE ,MEDICATION_DESCRIPTION ,CONDITION_CODE FROM MEDICATION_CODES WHERE CONDITION_CODE = `/message/conditionCode`</sql> </statement> <statement name="LABORATORY_TEST_CODES" ide:type="executeSql" iterate="conditionCode" ide:summary="Return all lab tests relevant to the condition from the LABORATORY_TEST_CODES table." ide:allowCompoundStatements="true"> <sql>SELECT LABORATORY_TEST_CODE ,LABORATORY_TEST_DESCRIPTION ,CONDITION_CODE FROM LABORATORY_TEST_CODES WHERE CONDITION_CODE = `/message/conditionCode`</sql> </statement> <statement name="dummy_record" ide:type="executeSql" ide:summary="This creates a dummy record in the output XML in the event none of the queries return a result." ide:allowCompoundStatements="true"> <sql>SELECT SYSDATETIME() AS 'dummy_time_record'</sql> </statement> </config></pre>		

Destinations

Name	Page
 no-op Meds and Labs from DB To Parsing Route..... No-operation	111



no-op REH to Sink
No-operation

Filter

On ECLIR - eCR Condition Checks route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - eCR Condition Checks Route Error Handler* to the *ECLIR - eCR Route Error Handler Sink* after the errors have been stored into the `eICR_ERROR` table in the ECLIR database.

Destinations

Name	Page
 ECLIR - eCR Route Error Handler Sink.....Sink	11

**Error Queue**

Filter

Error Queue

On ECLIR - eCR Condition Checks route

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------

**db eCR Condition Checks Route Errors**

Filter

Database LookUp

On ECLIR - eCR Condition Checks route

Notes

Purpose: This Database Lookup filter inserts an error record into the `eICR_ERROR` table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - eCR Condition Checks Route Error Handler*.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]

Configuration Properties	
Configuration File	
Message Type	Plain text
Definition	
Query Timeout	
Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0
Configuration File: Database LookUp 2.xml	
<pre> <statement name="Check for existing eICR_DOCUMENT_ID" ide:type="executeSql" test="Null(\$eICR_DOCUMENT_PARSED_ID)" ide:summary="Before creating a new record in eICR_DOCUMENT_PARSED, check if one exists already and use that ID to write out errors." ide:allowCompoundStatements="true"> <sql>SELECT ID AS eICR_DOCUMENT_PARSED_ID FROM eICR_DOCUMENT_PARSED WHERE DOCUMENT_OID = \$collectionID</sql> <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" test="NotNull(@eICR_DOCUMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table after eICR_DOCUMENT_PARSED_ID is found." ide:allowCompoundStatements="true"> <sql>INSERT INTO eICR_ERROR (eICR_DOCUMENT_PARSED_ID ,DOCUMENT_OID ,ERROR_CATEGORY ,ERROR_TYPE ,PROCESSING_STEP ,ERROR_MSG ,INSERT_DATE) VALUES (@eICR_DOCUMENT_PARSED_ID , \$collectionID , \$ERROR_CATEGORY , \$ERROR_TYPE , \$PROCESSING_STEP , \$ERROR_MSG , SYSDATETIME())</sql> </statement> <statement name="eICR_DOCUMENT_PARSED_ID Does Not exist" ide:type="executeSql" test="Null(@eICR_DOCUMENT_PARSED_ID)" ide:summary="The eICR_DOCUMENT_PARSED_ID does not exist in the eICR_DOCUMENT_PARSED table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true"> <sql>INSERT INTO eICR_DOCUMENT_PARSED (eICRDQ_RECORD_ID ,DOCUMENT_OID ,PLANNED_ORDERS_FL ,DOCUMENT_SOURCE_BUCKET ,INSERT_DATE) VALUES (\$eICRDQ_RECORD_ID , \$collectionID </pre>	

Configuration File: Database LookUp 2.xml

```

,0
,$DOCUMENT_SOURCE_BUCKET
,$SYSDATETIME()
)/<sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS eICR_DOCUMENT_PARSED_ID</sql>
    <substituteColumn name="eICR_DOCUMENT_PARSED_ID">$eICR_DOCUMENT_PARSED_ID</substitute
Column>
  </statement>
  <statement name="Insert into eICR_ERROR table" ide:type="executeSql" ide:summary="Add
error record to the eICR_ERROR table after eICR_DOCUMENT_PARSED_ID is created." ide:allowComp
oundStatements="true">
    <sql>INSERT INTO eICR_ERROR (
eICR_DOCUMENT_PARSED_ID
,$DOCUMENT_OID
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,$INSERT_DATE
)
VALUES (
@eICR_DOCUMENT_PARSED_ID
,$collectionID
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,$SYSDATETIME()
)/<sql>
  </statement>
  </statement>
  </statement>
  <statement name="Insert into eICR_ERROR table" ide:type="executeSql" test="NotNull($eICR_DOC
UMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table." ide:allowCompoundStat
ements="true">
    <sql>INSERT INTO eICR_ERROR (
eICR_DOCUMENT_PARSED_ID
,$DOCUMENT_OID
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,$INSERT_DATE
)
VALUES (
$eICR_DOCUMENT_PARSED_ID
,$collectionID
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,$SYSDATETIME()
)/<sql>
  </statement>
</config>

```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	106



db Lookup Condition Code ID from XML

Filter

Database Lookup

On ECLIR - eCR Condition Checks route

Notes

Purpose:

This Database Lookup filter iterates through each conditionCode element created in the XML output from the *js Build XML for db query* JavaScript filter.

This query returns the unique identity of each condition from the REPORTABLE_CONDITION_CODE table and sets the message property array Condition_ID_Pair to a concatenated value of the condition code + '==' + the reportable condition ID for each condition.

The eICR document is then sent to the *ECLIR - eICR Message Parsing* route and the message property array Condition_ID_Pair is used in the *js Parse eICR to XML for DB Inserts* JavaScript filter.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database Lookup.xml

Configuration File: Database LookUp.xml

```

<statement name="Lookup REPORTABLE_CONDITION_ID" ide:type="executeSql" iterate="condition
Code" ide:summary="Lookup the unique identity of the condition from the REPORTABLE_CONDITION
_CODE table and set the message property array Condition_ID_Pair to a concatenated value of the
condition code + '==' + the reportable condition ID." ide:allowCompoundStatements="true">
  <sql>SELECT TOP (1) CONCAT(`/message/conditionCode`, '==', ID) AS Condition_ID_Pair
FROM REPORTABLE_CONDITION
WHERE REPORTABLE_CONDITION_CODE = `/message/conditionCode`</sql>
  <substituteColumn name="Condition_ID_Pair">${Condition_ID_Pairs[]}</substituteColumn>
  <resultSet>AllowNullResultSet</resultSet>
</statement>
</config>

```

Destinations

Name	Page
 Snapshot Load eICR..... Snapshot Load	103

**Snapshot Save eICR****Snapshot Save**

On ECLIR - eCR Condition Checks route

Filter

Notes

Purpose: This Snapshot Save filter saves a copy of the eICR document to be restored later for use in the *ECLIR - eICR Message Parsing* route.

Destinations

Name	Page
 js Build XML for db query..... JavaScript (v.1)	103

**no-op Meds and Labs from DB To Parsing Route****No-operation**

On ECLIR - eCR Condition Checks route

Filter

Notes

Purpose: This No-operation filter sends the output from the *db Medications and Lab Results for Condition from XML Database Message Extraction* filter to the *ECLIR - eICR Message Parsing* route.

Destinations

Name	Page
 no-op Relevant Meds and Labs In.....No-operation	152



no-op eCR Msgs In

Filter

No-operation

On ECLIR - eCR Condition Checks route

Notes

Purpose: This No-operation filter receives eICR documents from the *ECLIR - eCR Messages In* route.

Destinations

Name	Page
 Snapshot Save eICR.....Snapshot Save	111



js Set Error properties

Filter

JavaScript (v.1)

On ECLIR - eCR Condition Checks route

Notes

Purpose: This JavaScript filter

Sets the property ERROR_CATEGORY with the type of error to be stored in the ERROR_CATEGORY column in the eICR_ERROR table. The category is set to 'FATAL' because the message stops processing.

Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the eICR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the eICR_ERROR table.

Sets the property ERROR_MSG with the Rhapsody generated errors to be stored in the ERROR_MSG column in the eICR_ERROR table.

Sets the property DOCUMENT_SOURCE_BUCKET based on the rhapsody:InputCommunicationPoint message property from the legacy routes to be stored in the ERROR_MSG column in the eICR_ERROR table.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
// _____
```

Javascript

```

//
=====

var next = output.append(input[0]);

//
=====
//_____ INITIAL VARIABLES (end
)_____
//
=====

//
=====
//_____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)_____
//
=====

next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - eCR Condition Checks (Route) ->' + next
.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
continue: ' + next.getErrors());
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
.getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);
//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR
_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}

//Set property for 'DOCUMENT_SOURCE_BUCKET' column based on the input comm point.
var DOCUMENT_SOURCE_BUCKET = next.getProperty('rhapsody:InputCommunicationPoint');
if (DOCUMENT_SOURCE_BUCKET == 'AIMS Platform') {
    DOCUMENT_SOURCE_BUCKET = '████_PROD';
}
else {
    DOCUMENT_SOURCE_BUCKET = '████_TEST';
}
next.setProperty('DOCUMENT_SOURCE_BUCKET', DOCUMENT_SOURCE_BUCKET);

//
=====
//_____ SET MESSAGE PROPERTIES FOR DB INSERT (end
)_____
//
=====

```

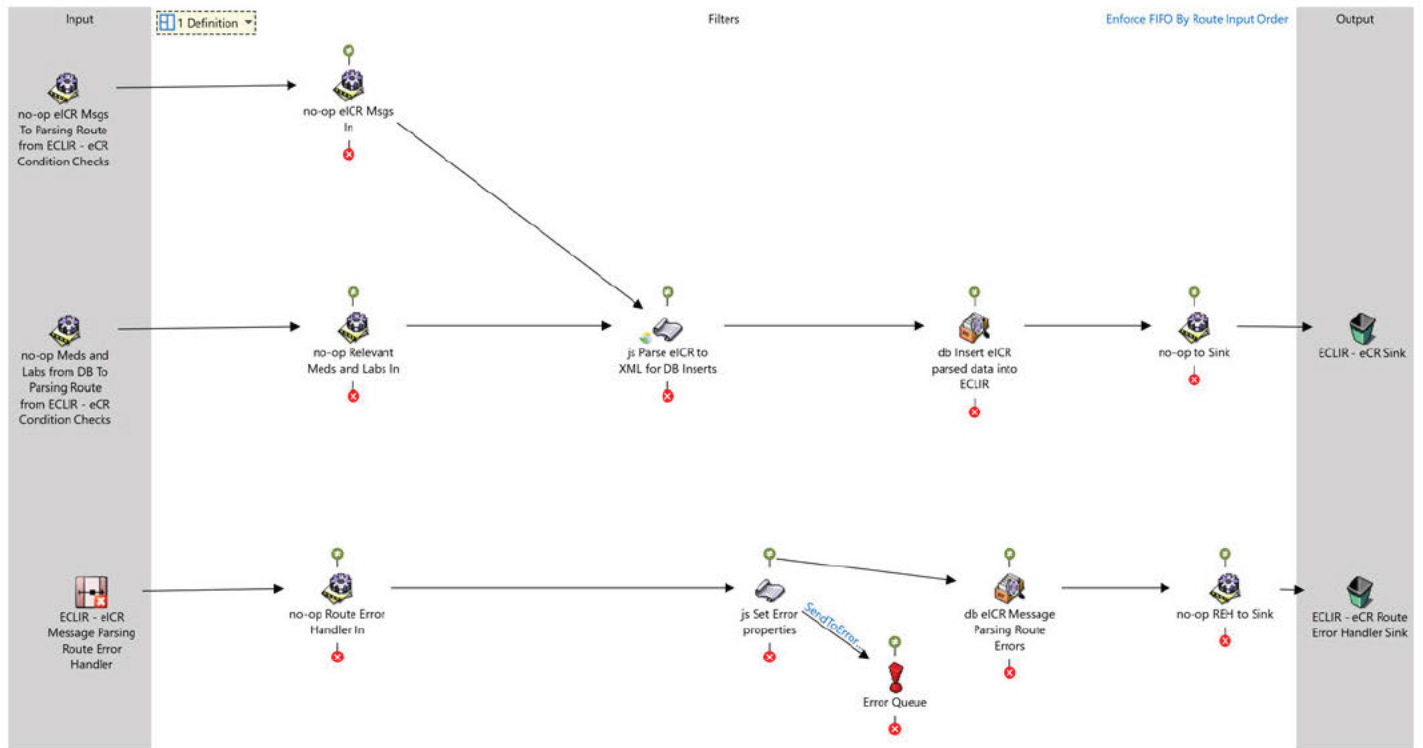
Destinations

Name		Page
Output Condition SendToErrorQueue = 'true' ignoring case		
 Error Queue.....	Error Queue	107
Error Connector		Page
 db eCR Condition Checks Route Errors.....	Database LookUp	107



ECLIR - eICR Message Parsing

Route



Notes

Purpose:

This route receives:

the eICR (Initial Case Report) document from the *ECLIR - eICR Condition Checks* route

the Database Message Extraction output XML for relevant medications and lab tests from the *ECLIR - eICR Condition Checks* route

The two messages are collected in the *js Parse eICR to XML for DB Inserts* JavaScript filter Message Collector based on the collectionID message property.

Both of these messages provide information to parse and create the XML output that will be used for the database inserts into the ECLIR eICR tables.

Inputs

no-op eICR Msgs To Parsing Route (filter from "ECLIR - eCR Condition Checks").....	No-operation	105
no-op Meds and Labs from DB To Parsing Route (filter from "ECLIR - eCR Condition Checks").....	No-operation	111

Outputs

ECLIR - eCR Sink.....	Sink	12
ECLIR - eCR Route Error Handler Sink.....	Sink	11

Filters

Name	Page
 db eICR Message Parsing Route Errors..... Database LookUp	117
 db Insert eICR parsed data into ECLIR..... Database LookUp	143
 Error Queue..... Error Queue	154
 js Parse eICR to XML for DB Inserts..... JavaScript (v.1)	119
 js Set Error properties..... JavaScript (v.1)	153
 no-op eICR Msgs In..... No-operation	152
 no-op REH to Sink..... No-operation	151
 no-op Relevant Meds and Labs In..... No-operation	152
 no-op Route Error Handler In..... No-operation	152
 no-op to Sink..... No-operation	155

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



db eICR Message Parsing Route Errors

Filter

Database LookUp

On ECLIR - eICR Message Parsing route

Notes

Purpose: This Database Lookup filter inserts an error record into the eICR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - eICR Message Parsing Route Error Handler*.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database LookUp 2.xml

```

<statement name="Check for existing eICR_DOCUMENT_ID" ide:type="executeSql" test="Null($eICR_DOCUMENT_PARSED_ID)" ide:summary="Before creating a new record in eICR_DOCUMENT_PARSED, check if one exists already and use that ID to write out errors." ide:allowCompoundStatements="true">
  <sql>SELECT ID AS eICR_DOCUMENT_PARSED_ID
  FROM eICR_DOCUMENT_PARSED
  WHERE DOCUMENT_OID = $collectionID</sql>
  <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" test="NotNull(@eICR_DOCUMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table after eICR_DOCUMENT_PARSED_ID is found." ide:allowCompoundStatements="true">
    <sql>INSERT INTO eICR_ERROR (
    eICR_DOCUMENT_PARSED_ID
    ,DOCUMENT_OID
    ,ERROR_CATEGORY
    ,ERROR_TYPE

```

Configuration File: Database LookUp 2.xml

```

,PROCESSING_STEP
,ERROR_MSG
,INSERT_DATE
)
VALUES (
  @eICR_DOCUMENT_PARSED_ID
  , $collectionID
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
)
</sql>
</statement>
<statement name="eICR_DOCUMENT_PARSED_ID Does Not exist" ide:type="executeSql" test="Null
(@eICR_DOCUMENT_PARSED_ID)" ide:summary="The eICR_DOCUMENT_PARSED_ID does not exist in the eICR
DOCUMENT_PARSED table. Create an entry in that table to create an ID." ide:allowCompoundStat
ements="true">
  <sql>INSERT INTO eICR_DOCUMENT_PARSED (
    eICRDQ_RECORD_ID
    ,DOCUMENT_OID
    ,PLANNED_ORDERS_FL
    ,DOCUMENT_SOURCE_BUCKET
    ,INSERT_DATE
  )
  VALUES (
    $eICRDQ_RECORD_ID
    , $collectionID
    , 0
    , $DOCUMENT_SOURCE_BUCKET
    , SYSDATETIME()
  )</sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS eICR_DOCUMENT_PARSED_ID</sql>
    <substituteColumn name="eICR_DOCUMENT_PARSED_ID">$eICR_DOCUMENT_PARSED_ID</substitute
Column>
  </statement>
  <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" ide:summary="Add
error record to the eICR_ERROR table after eICR_DOCUMENT_PARSED_ID is created." ide:allowComp
oundStatements="true">
    <sql>INSERT INTO eICR_ERROR (
      eICR_DOCUMENT_PARSED_ID
      ,DOCUMENT_OID
      ,ERROR_CATEGORY
      ,ERROR_TYPE
      ,PROCESSING_STEP
      ,ERROR_MSG
      ,INSERT_DATE
    )
    VALUES (
      @eICR_DOCUMENT_PARSED_ID
      , $collectionID
      , $ERROR_CATEGORY
      , $ERROR_TYPE
      , $PROCESSING_STEP
      , $ERROR_MSG
      , SYSDATETIME()
    )
  </sql>
</statement>

```

Configuration File: Database LookUp 2.xml

```

) </sql>
  </statement>
</statement>
<statement name="Insert into eICR_ERROR table" ide:type="executeSql" test="NotNull($eICR_DOCUMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table." ide:allowCompoundStatements="true">
  <sql>INSERT INTO eICR_ERROR (
    eICR_DOCUMENT_PARSED_ID
  ,DOCUMENT_OID
  ,ERROR_CATEGORY
  ,ERROR_TYPE
  ,PROCESSING_STEP
  ,ERROR_MSG
  ,INSERT_DATE
  )
  VALUES (
    $eICR_DOCUMENT_PARSED_ID
  , $collectionID
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
  ) </sql>
</statement>
</config>

```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	151

**js Parse eICR to XML for DB Inserts**

Filter

JavaScript (v.1)

On ECLIR - eICR Message Parsing route

Notes

Purpose: This JavaScript filter parses data from the eICR document to be written to an XML file that follows the ECLIR_eCR_Parsing_DB_schema.xsd format for the *db Insert eICR parsed data into ECLIR* Database Lookup filter to use to iterate through each element corresponding to the ECLIR tables to be written to.

A message collector is used to input 2 messages into the filter.

The first message is the original eICR document with the REPORTABLE_CONDITION_ID's set in the message property array Condition_ID_Pairs for the triggered condition(s).

The second message is the XML message created from the queries to return lab results and medications relevant to the triggered condition(s).

If medications or lab tests relevant to the triggered condition(s) are found in the eICR document from the XML list from the second input, these are written to the ECLIR database. All irrelevant medications and lab results are not recorded in the database.

Message properties previously created in the *ECLIR - eCR Messages In* route from the RR are also written to the output XML to be stored in ECLIR tables.

Notes

LP Created filter 9/10/2024

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
next.text = next.text.replace(/\<ClinicalDocument (.*)\>/,
'<ClinicalDocument xmlns="urn:hl7-org:v3" xmlns:sdct="urn:hl7-org:sdct" xmlns:sdctxmlnamespaceholder="urn:hl7-org:v3" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="urn:hl7-org:v3 CDA_SDTC.xsd">');
next.setText(next.text, "UTF8");
var qryConditionMedsAndLabs = input[1];
var dbInsertsXML = new XML(<message></message>);
var tempEle = '';
var sectionEncounters = '/ClinicalDocument/component/structuredBody/component/section[code/@code="46240-8" or templateId/@root="2.16.840.1.113883.10.20.22.2.22" or templateId/@root="2.16.840.1.113883.10.20.22.2.22.1"]';
var sectionHistoryOfPresentIllness = '/ClinicalDocument/component/structuredBody/component/section[code/@code="10164-2" or templateId/@root="1.3.6.1.4.1.19376.1.5.3.1.3.4"]';
var sectionImmunizations = '/ClinicalDocument/component/structuredBody/component/section[code/@code="11369-6" or templateId/@root="2.16.840.1.113883.10.20.22.2.2" or templateId/@root="2.16.840.1.113883.10.20.22.2.2.1"]';
var sectionMedications = '/ClinicalDocument/component/structuredBody/component/section[code/@code="29549-3" or templateId/@root="2.16.840.1.113883.10.20.22.2.38"]';
var sectionPlanOfTreatment = '/ClinicalDocument/component/structuredBody/component/section[code/@code="18776-5" or templateId/@root="2.16.840.1.113883.10.20.22.2.10"]';
var sectionProblems = '/ClinicalDocument/component/structuredBody/component/section[code/@code="11450-4" or templateId/@root="2.16.840.1.113883.10.20.22.2.5" or templateId/@root="2.16.840.1.113883.10.20.22.2.5.1"]';
var sectionReasonForVisit = '/ClinicalDocument/component/structuredBody/component/section[code/@code="29299-5" or templateId/@root="2.16.840.1.113883.10.20.22.2.12"]';
var sectionResults = '/ClinicalDocument/component/structuredBody/component/section[code/@code="30954-2" or templateId/@root="2.16.840.1.113883.10.20.22.2.3" or templateId/@root="2.16.840.1.113883.10.20.22.2.3.1"]';
var sectionSocialHistory = '/ClinicalDocument/component/structuredBody/component/section[code/@code="29762-2" or templateId/@root="2.16.840.1.113883.10.20.22.2.17"]';
var sectionPregnancy = '/ClinicalDocument/component/structuredBody/component/section[code/@code="90767-5" or templateId/@root="2.16.840.1.113883.10.20.22.2.80"]';

next.indexInputProperty('Condition_ID_Pairs');
next.indexOutputProperty('Condition_ID_Pairs');
next.indexInputProperty('collectionID');
next.indexOutputProperty('collectionID');

//Set property for 'DOCUMENT_SOURCE_BUCKET' column based on the input comm point.
var DOCUMENT_SOURCE_BUCKET = next.getProperty('rhapsody:InputCommunicationPoint');
if (DOCUMENT_SOURCE_BUCKET == 'AIMS Platform') {
    DOCUMENT_SOURCE_BUCKET = '███_PROD';
}
else {
    DOCUMENT_SOURCE_BUCKET = '███_TEST';
}
```

Javascript

```

}
next.setProperty('DOCUMENT_SOURCE_BUCKET', DOCUMENT_SOURCE_BUCKET);

//
=====
//_____INITIAL VARIABLES (end
)_____
//
=====

//
=====
//_____FUNCTIONS (start
)_____
//
=====
if (true) {
//FUNCTION: fnDBColumnInsert
//Function to insert node into output XML.
function fnDBColumnInsert(nameValue, columnValue) {
  if (columnValue != null && columnValue != 'null') {
    var dbColumnName = <{nameValue}>{columnValue}</{nameValue}>;
    tempEle.appendChild(dbColumnName);
  }
}

//FUNCTION: fnTruncateToMax
//Function to truncate the strField to the max length intMax.
function fnTruncateToMax(strFieldPath, intMax) {
  var strField = null;

  if (strFieldPath.indexOf('||') == -1) {
    if (next.getField(strFieldPath) != null) {
      strField = next.getField(strFieldPath).substring(0, intMax);
    }
  }
  else {
    if (strFieldPath != null) {
      var arrFieldPaths = strFieldPath.split('||');
      for (var i = 0; i < arrFieldPaths.length; i++) {
        if (next.getField(arrFieldPaths[i]) != null) {
          strField = next.getField(arrFieldPaths[i]).substring(0, intMax);
          break;
        }
      }
    }
  }
  return strField;
}

//FUNCTION: fnConcatValues
//Function to concatenate the values in strField delimited with strDelimiter.
function fnConcatValues(strFieldPath, strDelimiter) {

```

Javascript

```

var strField = null;

if (strFieldPath != null) {
    var arrFieldPaths = strFieldPath.split('||');
    for (var i = 0; i < arrFieldPaths.length; i++) {
        if (next.getField(arrFieldPaths[i]) != null) {
            if (strField == null) { strField = next.getField(arrFieldPaths[i]); }
            else { strField += strDelimiter + next.getField(arrFieldPaths[i]); }
        }
    }
}
return strField;
}

//FUNCTION: fnCreateTelecomNode
//--- Function to create XML node for phone and email data.
function fnCreateTelecomNode(strFieldPath, strEmailColumn, intEmailMax, strPhoneColumn, int
PhoneMax) {
    var telecomPhone = null;
    var telecomPhone_formatted = null;
    var telecomEmail = null;

    if (strFieldPath != null) {
        var arrFieldPaths = strFieldPath.split('||');
        for (var i = 0; i < arrFieldPaths.length; i++) {
            if (next.getFieldAsList(arrFieldPaths[i]) != null) {
                var arrTelecom = next.getFieldAsList(arrFieldPaths[i]);
                for (var j = 0; j < arrTelecom.length; j++) {
                    var telecomValue = arrTelecom[j];
                    if (telecomValue != null) {
                        if (telecomPhone == null && telecomValue.indexOf('tel:') != -1) {
                            telecomPhone = telecomValue.substring(telecomValue.indexOf('tel:') + 4);
                            var cleaned = ('' + telecomPhone).replace(/\D/g, '');
                            var match = cleaned.match(/^(\d)?(\d{3})(\d{3})(\d{4})$/);
                            if (match) {
                                var intlCode = (match[1] ? '+1 ' : '');
                                telecomPhone_formatted = ['(', match[2], ')', match[3], '-', match[4]].join('');
                            }
                        }

                        if (telecomEmail == null && telecomValue.indexOf('mailto:') != -1) {
                            telecomEmail = telecomValue.substring(telecomValue.indexOf('mailto:') + 7);
                        }
                    }
                }
            }
        }
    }

    if (telecomPhone != null) { fnDBColumnInsert(strPhoneColumn, telecomPhone.substring(0, int
PhoneMax)); }
    if (telecomPhone_formatted != null) { fnDBColumnInsert(strPhoneColumn+'_FORMATTED', telecom
Phone_formatted.substring(0, intPhoneMax)); }
    if (telecomEmail != null) { fnDBColumnInsert(strEmailColumn, telecomEmail.substring(0, int
EmailMax)); }
}

```

Javascript

```

//FUNCTION: fnCurrentlyPregnant
//--- Function to determine if the parsed pregnancy data indicates the patient was pregnant at
the time of the document generation.
function fnCurrentlyPregnant() {
  //Populated with a pregnancy status with the strFieldPath formatted as a date YYYYMMDD.
  var isPregnant = false;
  var isPregnancyPresent = false; //If not present, don't map the question.
  var strFieldPath = sectionSocialHistory + '/entry/observation[value/@code="77386006"]/eff
ectiveTime/low/@value'
    + '||' + sectionProblems + '/entry/act/entryRelationship/observation[value/@code="773
86006"]/effectiveTime/low/@value'
    + '||' + sectionPregnancy + '/entry/observation[value/@code="77386006"]/effectiveTime/
low/@value';
  if (strFieldPath != null) {
    var arrFieldPaths = strFieldPath.split('||');
    for (var indexPath = 0; indexPath < arrFieldPaths.length; indexPath++) {
      if (next.getField(arrFieldPaths[indexPath]) != null) {
        try {
          isPregnancyPresent = true;
          var startDate = next.getField(arrFieldPaths[indexPath]);
          if (startDate.length >= 8) {
            startDate = new Date(Number(startDate.substr(0,4)),Number(startDate.substr(4,2))-1,Number
(startDate.substr(6,2)));
            // Calculate the expected due date (40 weeks after the start date)
            var dueDate = new Date(startDate);
            dueDate.setDate(dueDate.getDate() + (40 * 7));
            var documentDate = next.getField('/ClinicalDocument/effectiveTime/@value');
            documentDate = new Date(Number(documentDate.substr(0,4)),Number(documentDate.substr(4,2
))-1,Number(documentDate.substr(6,2)));
            // Check if the document date is after the start date and before the due date
            if (documentDate >= startDate && documentDate <= dueDate) {
              isPregnant = true;
            }
          }
        } catch(err) {
          //If the low value date isn't valid and cannot be processed, ignore.
        }
      }
    }
  }

  if (isPregnant) { return 'YES'; }
  else { return 'NO'; }
}

//FUNCTION: fnIsRelevantMedication
//Function to check if a medication is relevant to the triggered condition by checking the xml
returned from the database in the ECLIR - eICR Condition Checks route.
function fnIsRelevantMedication (strCondition, strMed) {
  if (qryConditionMedsAndLabs.getField('/message/MEDICATION_CODES[MEDICATION_CODE="' + strMed +
'" and CONDITION_CODE="' + strCondition + '"]/MEDICATION_CODE') != null) {
    return true;
  }
  else { return false; }
}

```

Javascript

```

//FUNCTION: fnIsRelevantLabTest
//Function to check if a lab test is relevant to the triggered condition by checking the xml
//returned from the database in the ECLIR - eICR Condition Checks route.
function fnIsRelevantLabTest (strCondition, strTest) {
  if (qryConditionMedsAndLabs.getField('/message/LABORATORY_TEST_CODES[LABORATORY_TEST_CODE="' +
    strTest + '" and CONDITION_CODE="' + strCondition + '"]/LABORATORY_TEST_CODE') != null) {
    return true;
  }
  else { return false; }
}

//FUNCTION: fnConditionCodeID
//Function to get the Condition Code ID from a list of value pairs in the property array 'Con
//dition_ID_Pairs'.
function fnConditionCodeID(indexConditionArray) {
  var arrConditionCodeIDs = next.getPropertyAsList('Condition_ID_Pairs');
  var Condition_ID_Pair = arrConditionCodeIDs.get(indexConditionArray);
  var ConditionCodeID = null;

  if (Condition_ID_Pair != null) {
    var arrSplitPair = Condition_ID_Pair.split('==');
    if ((arrSplitPair[1] != null && arrSplitPair[1] != '' && typeof(arrSplitPair[1]) != 'und
efined')) {
      ConditionCodeID = arrSplitPair[1];
    }
  }

  return ConditionCodeID;
}

//FUNCTION: fnGetDocTriggerValue
//Function to get the eICR_DOCUMENT_TRIGGER column values from each property set in the
// ECLIR - eICR Messages In route by condition.
function fnGetDocTriggerValue(propertyName, propertyIndex, intMax) {
  var propertyValue = next.getProperty(propertyName);
  var strValue = null;
  if (propertyValue != null) {
    var arrValues = propertyValue.split('^');
    for (var i = 0; i < arrValues.length; i++) {
      if (arrValues[propertyIndex] != null) {
        strValue = arrValues[propertyIndex].substring(arrValues[propertyIndex].indexOf('==')+2);
        if (strValue != null || strValue != 'null') { strValue = strValue.substring(0, intMax); }
      }
    }
  }
  return strValue;
}

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
  num = num.toString();

```

Javascript

```

while (num.length < size) num = "0" + num;
return num;
}

//FUNCTION: fnGetDateTimeComponents
//--- Function to extract individual components from a Date/Time value.
function fnGetDateTimeComponents (inputDateTime) {
    var inputYear = '';
    var inputMonth = '';
    var inputDay = '';
    var inputHours = '';
    var inputMinutes = '';
    var inputSeconds = '';
    var inputOffsetHours = 0;
    var inputOffsetMinutes = 0;
    var inputOffsetSign = '';
    if (inputDateTime != null) {
        if (inputDateTime.length >= 8) {
            if (inputDateTime.match(/^((?:((?:((?:[13579][26]|2468)[048])00)|(?:[0-9]{2}((?:[13579][26])|(?:[2468][048]|0[48]))))|(?:((?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30))|(?:((?:01|03|05|07|08|10|12)(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:0[1-9]|1[0-9]|2[0-9]))))|(?:[0-9]{4}((?:((?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30))|(?:((?:01|03|05|07|08|10|12)(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:[01][0-9]|2[0-8])))))(?:0[0-9]|1[0-9]|2[0-3]))?(?:[0-5][0-9])?2}?)?.\d)?\d?\d?\d?((\+|-)[0-1][0-9]{3})?$/)) != null) {
                inputYear = fnPadZeros(inputDateTime.slice(0, 4), 4);
                inputMonth = fnPadZeros(inputDateTime.slice(4, 6), 2);
                inputDay = fnPadZeros(inputDateTime.slice(6, 8), 2);
                inputHours = fnPadZeros(inputDateTime.slice(8, 10), 2);
                inputMinutes = fnPadZeros(inputDateTime.slice(10, 12), 2);
                inputSeconds = fnPadZeros(inputDateTime.slice(12, 14), 2);
                indexOffsetSign = inputDateTime.lastIndexOf('+');
                if (indexOffsetSign == -1) { indexOffsetSign = inputDateTime.lastIndexOf('-'); }

                if (indexOffsetSign != -1) { //formats with time-zone offsets
                    inputOffsetSign = inputDateTime.substr(indexOffsetSign, 1);
                    indexOffsetSign++;
                    inputOffsetHours = inputDateTime.substr(indexOffsetSign, 2);
                    indexOffsetSign = indexOffsetSign+2;
                    inputOffsetMinutes = inputDateTime.substr(indexOffsetSign, 2);
                }
            }
        }
    }
    return {
        strYear: inputYear,
        strMonth: inputMonth,
        strDay: inputDay,
        strHours: inputHours,
        strMinutes: inputMinutes,
        strSeconds: inputSeconds,
        offsetSign: inputOffsetSign,
        offsetHours: inputOffsetHours,
        offsetMinutes: inputOffsetMinutes
    }
}

```

Javascript

```

//FUNCTION: fnConvertToDocumentTimeZone
//Function to convert Date/Time fields to the same time zone as the document time.
function fnConvertToDocumentTimeZone(inYear, inMonth, inDay, inHours, inMinutes, inSeconds, in
OffsetSign, inOffsetHours, inOffsetMinutes) {
    var inputOffsetMilliseconds = (Number(inOffsetHours) * 60 + Number(inOffsetMinutes)) * 60000;
    if (inOffsetSign == '-') { inputOffsetMilliseconds = inputOffsetMilliseconds * -1; }

    if (next.getField('/ClinicalDocument/effectiveTime/@value') != null) {
        var documentDateTimeInfo = fnGetDateTimeComponents (next.getField('/ClinicalDocument/eff
ectiveTime/@value'));
        var documentOffsetMilliseconds = (Number(documentDateTimeInfo.offsetHours) * 60 + Number(doc
umentDateTimeInfo.offsetMinutes)) * 60000;
        if (documentDateTimeInfo.offsetSign == '-') { documentOffsetMilliseconds = documentOffset
Milliseconds * -1; }

        // Calculate the difference in offsets
        var offsetDifference = documentOffsetMilliseconds - inputOffsetMilliseconds;
        // Convert the date to the reference time zone
        var convertedDate = new Date(new Date(inYear, inMonth, inDay, inHours, inMinutes, inSeconds
).getTime() + offsetDifference);
    }
    return convertedDate;
}

//FUNCTION: fnConvertDateTimeField
//Function to convert Date/Time fields to a format compatible with the database (YYYY-MM-DD HH
:MI:SS).
function fnConvertDateTimeField(fieldDateTime) {
    var formattedFieldDateTime = fieldDateTime;
    if (fieldDateTime != null) {
        var fieldComponents = fnGetDateTimeComponents (fieldDateTime);
        var strAdjustedFieldDateTime = new Date(fnConvertToDocumentTimeZone(
            fieldComponents.strYear,
            fieldComponents.strMonth-1,
            fieldComponents.strDay,
            fieldComponents.strHours,
            fieldComponents.strMinutes,
            fieldComponents.strSeconds,
            fieldComponents.offsetSign,

            fieldComponents.offsetHours,
            fieldComponents.offsetMinutes));
        var fieldYear = strAdjustedFieldDateTime.getFullYear();
        var fieldMonth = fnPadZeros(strAdjustedFieldDateTime.getMonth() + 1, 2);
        var fieldDay = fnPadZeros(strAdjustedFieldDateTime.getDate(), 2);
        var fieldHours = fnPadZeros(strAdjustedFieldDateTime.getHours(), 2);
        var fieldMinutes = fnPadZeros(strAdjustedFieldDateTime.getMinutes(), 2);
        var fieldSeconds = fnPadZeros(strAdjustedFieldDateTime.getSeconds(), 2);

        // Format the date string
        formattedFieldDateTime = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+field
Minutes+':'+fieldSeconds;
    }
    return formattedFieldDateTime;
}

//FUNCTION: fnConvertDOBField

```

Javascript

```

//Function to convert Date/Time Date of Birth field to a format compatible with the database
(YYYY-MM-DD HH:MI:SS). This function does not consider time zones.
function fnConvertDOBField(fieldDateTime) {
    var formattedFieldDateTime = fieldDateTime;
    if (fieldDateTime != null) {
        var fieldComponents = fnGetDateTimeComponents (fieldDateTime);

        // Format the date string
        formattedFieldDateTime = fieldComponents.strYear+'-'+fieldComponents.strMonth+'-'+fieldComp
onents.strDay+' '+fieldComponents.strHours+':'+fieldComponents.strMinutes+':'+fieldComponents
.strSeconds;
    }
    return formattedFieldDateTime;
}

//FUNCTION: fnGetMostRecentAddress
//Function to get most recent address based on the useablePeriod/low/@value.
function fnGetMostRecentAddress(strFieldPath) {
    var cntAddresses = next.getRepeatCount(strFieldPath);
    var mostRecentAddressDate = '';
    var mostRecentAddressDateIndex = 0;
    var noDateAddressIndex = '';

    for (var i=0; i < cntAddresses; i++) {
        if (next.getField(strFieldPath + '['+(i+1)+']/useablePeriod/low/@value') != null) {
            if (mostRecentAddressDate == '') {
                mostRecentAddressDateIndex = (i+1);
                mostRecentAddressDate = Number(next.getField(strFieldPath + '['+(i+1)+']/useablePeriod/low/
@value'));
            }
            if (Number(next.getField(strFieldPath + '['+(i+1)+']/useablePeriod/low/@value')) > Number
(mostRecentAddressDate)) {
                mostRecentAddressDateIndex = (i+1);
                mostRecentAddressDate = Number(next.getField(strFieldPath + '['+(i+1)+']/useablePeriod/low/
@value'));
            }
        }
        //Get index of first address with no usablePeriod value.
        else {
            if (noDateAddressIndex == '') {
                noDateAddressIndex = (i+1);
            }
        }
    }
    if (mostRecentAddressDate != '') {
        //If there is more than one address and one of the fields has no useablePeriod, check the
most recent address for a high/@value.
        //If there is a high/@value, use the address with no date.
        if ((noDateAddressIndex != '') && (next.getField(strFieldPath + '['+mostRecentAddressDate
Index+']/useablePeriod/high/@value') != null)) {
            mostRecentAddressDateIndex = noDateAddressIndex;
        }
    }
    else {
        //No addresses found with useablePeriod/low/@value so use the first address if there is one.
        mostRecentAddressDateIndex = noDateAddressIndex;
    }
}

```

Javascript

```

var PATIENT_ADDRESS_LINE = next.getField(strFieldPath + '['+mostRecentAddressDateIndex+']/streetAddressLine[1]');
if (next.getField(strFieldPath + '['+mostRecentAddressDateIndex+']/streetAddressLine[2]') != null) {
    PATIENT_ADDRESS_LINE += ', ' + next.getField(strFieldPath + '['+mostRecentAddressDateIndex+']/streetAddressLine[2]');
}
fnDBColumnInsert('PATIENT_ADDRESS_LINE', PATIENT_ADDRESS_LINE, 500);
fnDBColumnInsert('PATIENT_CITY', fnTruncateToMax(strFieldPath + '['+mostRecentAddressDateIndex+']/city', 150));
fnDBColumnInsert('PATIENT_STATE', fnTruncateToMax(strFieldPath + '['+mostRecentAddressDateIndex+']/state', 150));
fnDBColumnInsert('PATIENT_ZIP', fnTruncateToMax(strFieldPath + '['+mostRecentAddressDateIndex+']/postalCode', 50));
fnDBColumnInsert('PATIENT_COUNTY', fnTruncateToMax(strFieldPath + '['+mostRecentAddressDateIndex+']/county', 250));
fnDBColumnInsert('PATIENT_COUNTRY', fnTruncateToMax(strFieldPath + '['+mostRecentAddressDateIndex+']/country', 1000));
}
//
=====
// _____ FUNCTIONS (end
)
//
=====
}

//
=====
// _____ SET eICR_DOCUMENT_PARSED XML NODES (start
)
//
=====
// Create XML nodes for key fields from the eICR document to be written to the eICR_DOCUMENT
_PARSED table.

tempEle = new XML(<eICR_DOCUMENT_PARSED></eICR_DOCUMENT_PARSED>);

var encompassingEncounter = '/ClinicalDocument/componentOf/encompassingEncounter';

fnDBColumnInsert('DOCUMENT_OID', next.getProperty('collectionID'));
fnDBColumnInsert('DOC_ID_ROOT', fnTruncateToMax('/ClinicalDocument/id/@root', 200));
fnDBColumnInsert('DOC_ID_EXTENSION', fnTruncateToMax('/ClinicalDocument/id/@extension', 200));

var SET_ID_CONCAT = fnConcatValues('/ClinicalDocument/setId/@root||/ClinicalDocument/setId/@extension', '&');
if (SET_ID_CONCAT != null) { fnDBColumnInsert('SET_ID_CONCAT', SET_ID_CONCAT.substring(0, 500)); }

fnDBColumnInsert('SET_ID_ROOT', fnTruncateToMax('/ClinicalDocument/setId/@root', 200));
fnDBColumnInsert('SET_ID_EXTENSION', fnTruncateToMax('/ClinicalDocument/setId/@extension', 200));
fnDBColumnInsert('DOCUMENT_VERSION', fnTruncateToMax('/ClinicalDocument/versionNumber/@value', 250));

```

Javascript

```
//ecrNOW FHIR app implementers will send this in the displayName attribute.
if (next.getField('/ClinicalDocument/author/assignedAuthor/assignedAuthoringDevice/manufacture
ModelName/@displayName') != null) {
    var arrEHR_VENDOR = next.getFieldAsList('/ClinicalDocument/author/assignedAuthor/assignedAuth
oringDevice/manufactureModelName/@displayName');
    var arrEHR_VENDOR_SOFTWARE_VERSION = next.getFieldAsList('/ClinicalDocument/author/assigned
Author[assignedAuthoringDevice/manufactureModelName/@displayName]/assignedAuthoringDevice/sof
twareName/@displayName');
}
else if (next.getField('/ClinicalDocument/author/assignedAuthor/assignedAuthoringDevice/manufac
turerModelName') != null) {
    var arrEHR_VENDOR = next.getFieldAsList('/ClinicalDocument/author/assignedAuthor/assignedAuth
oringDevice/manufactureModelName');
    var arrEHR_VENDOR_SOFTWARE_VERSION = next.getFieldAsList('/ClinicalDocument/author/assigned
Author[assignedAuthoringDevice/manufactureModelName]/id/@extension');
}
if (arrEHR_VENDOR != null) {
    for (var i = 0; i < arrEHR_VENDOR.length; i++) {
        if (arrEHR_VENDOR[i] != null) { fnDBColumnInsert('EHR_VENDOR_'+(i+1), arrEHR_VENDOR[i].sub
string(0, 500)); }
    }
}

if (arrEHR_VENDOR_SOFTWARE_VERSION != null) {
    for (var i = 0; i < arrEHR_VENDOR_SOFTWARE_VERSION.length; i++) {
        if (arrEHR_VENDOR_SOFTWARE_VERSION[i] != null) { fnDBColumnInsert('EHR_VENDOR_SOFTWARE_VER
SION_'+(i+1), arrEHR_VENDOR_SOFTWARE_VERSION[i].substring(0, 150)); }
    }
}

fnDBColumnInsert('DOCUMENT_CREATE_DATE', fnTruncateToMax('/ClinicalDocument/effectiveTime/
@value', 100));
fnDBColumnInsert('EVENT_DATE_CONVERTED', fnConvertDateTimeField(next.getField('/ClinicalDocu
ment/effectiveTime/@value')));
fnDBColumnInsert('ENCOUNTER_DATE_LOW', fnTruncateToMax(encompassingEncounter + '/effectiveTime/
low/@value', 100));
fnDBColumnInsert('ENCOUNTER_DATE_LOW_CONVERTED', fnConvertDateTimeField(next.getField(encompa
ssingEncounter + '/effectiveTime/low/@value')));
fnDBColumnInsert('ENCOUNTER_DATE_HIGH', fnTruncateToMax(encompassingEncounter + '/effectiveTime
/high/@value', 100));
fnDBColumnInsert('ENCOUNTER_DATE_HIGH_CONVERTED', fnConvertDateTimeField(next.getField(encompa
ssingEncounter + '/effectiveTime/high/@value')));
fnDBColumnInsert('ENCOMPASSING_ENCOUNTER_CODE', fnTruncateToMax(encompassingEncounter + '/code/
@code', 250));
fnDBColumnInsert('CUSTODIAN_ORG_ROOT_VALUE', fnTruncateToMax('/ClinicalDocument/custodian/ass
ignedCustodian/representedCustodianOrganization/id/@root', 250));
fnDBColumnInsert('CUSTODIAN_ORG_EXTENSION_VALUE', fnTruncateToMax('/ClinicalDocument/custodian/
assignedCustodian/representedCustodianOrganization/id/@extension', 250));
fnDBColumnInsert('CUSTODIAN_ORG_NAME', fnTruncateToMax('/ClinicalDocument/custodian/assigned
Custodian/representedCustodianOrganization/name', 500));
fnDBColumnInsert('CUSTODIAN_ORG_ADDRESS_LINE', fnTruncateToMax('/ClinicalDocument/custodian/ass
ignedCustodian/representedCustodianOrganization/addr/streetAddressLine', 500));
fnDBColumnInsert('CUSTODIAN_ORG_CITY', fnTruncateToMax('/ClinicalDocument/custodian/assigned
Custodian/representedCustodianOrganization/addr/city', 150));
fnDBColumnInsert('CUSTODIAN_ORG_STATE', fnTruncateToMax('/ClinicalDocument/custodian/assigned
Custodian/representedCustodianOrganization/addr/state', 150));
fnDBColumnInsert('CUSTODIAN_ORG_ZIP', fnTruncateToMax('/ClinicalDocument/custodian/assignedCust
odian/representedCustodianOrganization/addr/postalCode', 150));
```

Javascript

```

fnDBColumnInsert('CUSTODIAN_ORG_COUNTY', fnTruncateToMax('/ClinicalDocument/custodian/assigned
Custodian/representedCustodianOrganization/addr/county', 250));

fnCreateTelecomNode('/ClinicalDocument/custodian/assignedCustodian/representedCustodianOrganiza
tion/telecom/@value'+''')
    +encompassingEncounter + '/responsibleParty/assignedEntity/telecom/@value', 'CUSTODIAN
_ORG_EMAIL', 250, 'CUSTODIAN_ORG_PHONE', 250);

fnDBColumnInsert('SERVICE_PROVIDER_NAME', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/serviceProviderOrganization/name', 500));
fnDBColumnInsert('SERVICE_PROVIDER_ADDRESS_LINE', fnTruncateToMax(encompassingEncounter + '/loc
ation/healthCareFacility/serviceProviderOrganization/addr/streetAddressLine', 500));
fnDBColumnInsert('SERVICE_PROVIDER_CITY', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/serviceProviderOrganization/addr/city', 150));
fnDBColumnInsert('SERVICE_PROVIDER_STATE', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/serviceProviderOrganization/addr/state', 150));
fnDBColumnInsert('SERVICE_PROVIDER_ZIP', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/serviceProviderOrganization/addr/postalCode', 50));
fnDBColumnInsert('SERVICE_PROVIDER_COUNTY', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/serviceProviderOrganization/addr/county', 250));
fnDBColumnInsert('HEALTHCARE_FACILITY_NAME', fnTruncateToMax(encompassingEncounter + '/location
/healthCareFacility/location/name', 500));
fnDBColumnInsert('HEALTHCARE_FACILITY_DISPLAY', fnTruncateToMax(encompassingEncounter + '/loc
ation/healthCareFacility/code/@displayName', 500));
fnDBColumnInsert('HEALTHCARE_FACILITY_ADDRESS_LINE', fnTruncateToMax(encompassingEncounter + '/
location/healthCareFacility/location/addr/streetAddressLine', 500));
fnDBColumnInsert('HEALTHCARE_FACILITY_CITY', fnTruncateToMax(encompassingEncounter + '/location
/healthCareFacility/location/addr/city', 150));
fnDBColumnInsert('HEALTHCARE_FACILITY_STATE', fnTruncateToMax(encompassingEncounter + '/loc
ation/healthCareFacility/location/addr/state', 150));
fnDBColumnInsert('HEALTHCARE_FACILITY_ZIP', fnTruncateToMax(encompassingEncounter + '/location/
healthCareFacility/location/addr/postalCode', 50));
fnDBColumnInsert('HEALTHCARE_FACILITY_COUNTY', fnTruncateToMax(encompassingEncounter + '/loc
ation/healthCareFacility/serviceProviderOrganization/addr/county', 250));
fnDBColumnInsert('PROVIDER_FIRST_NAME', fnTruncateToMax(encompassingEncounter + '/responsible
Party/assignedEntity/assignedPerson/name/given', 250));
fnDBColumnInsert('PROVIDER_LAST_NAME', fnTruncateToMax(encompassingEncounter + '/responsible
Party/assignedEntity/assignedPerson/name/family', 250));
fnDBColumnInsert('PROVIDER_ADDRESS_LINE', fnTruncateToMax(encompassingEncounter + '/responsible
Party/assignedEntity/representedOrganization/addr/streetAddressLine', 500));
fnDBColumnInsert('PROVIDER_CITY', fnTruncateToMax(encompassingEncounter + '/responsibleParty/
assignedEntity/representedOrganization/addr/city', 150));
fnDBColumnInsert('PROVIDER_STATE', fnTruncateToMax(encompassingEncounter + '/responsibleParty/
assignedEntity/representedOrganization/addr/state', 150));
fnDBColumnInsert('PROVIDER_ZIP', fnTruncateToMax(encompassingEncounter + '/responsibleParty/ass
ignedEntity/representedOrganization/addr/postalCode', 50));

//Reason for visit section
// Extract the text in the ReasonForVisit section and concatenate values with a ';'.
var REASON_FOR_VISIT = '';
var indexRFVsection = next.text.indexOf('29299-5');
if (indexRFVsection == -1) {
    indexRFVsection = next.text.indexOf('2.16.840.1.113883.10.20.22.2.12');
}

if (next.getField(sectionReasonForVisit + '/text') != null) {
    var indexRFVText_begin = next.text.indexOf('<text>', indexRFVsection);
    var indexRFVText_end = next.text.indexOf('</text>', indexRFVText_begin);

```

Javascript

```

indexRFVText_end = indexRFVText_end + 7;
REASON_FOR_VISIT = next.text.substring(indexRFVText_begin, indexRFVText_end);
}
else if (next.getField(sectionReasonForVisit + '/entry/observation/value/originalText') !=
null) {
var indexRFVText_begin = next.text.indexOf('<observation>', indexRFVsection);
indexRFVText_begin = next.text.indexOf('<value>', indexRFVText_begin);
indexRFVText_begin = next.text.indexOf('<originalText>', indexRFVText_begin);
var indexRFVText_end = next.text.indexOf('</originalText>', indexRFVText_begin);
indexRFVText_end = indexRFVText_end + 15;
REASON_FOR_VISIT = next.text.substring(indexRFVText_begin, indexRFVText_end);
}

var indexSearchEnd = REASON_FOR_VISIT.length;

if (REASON_FOR_VISIT != '') {
var strRFVConcat = '';
var indexCurrent = REASON_FOR_VISIT.indexOf('>')+1;
while (indexCurrent < indexSearchEnd) {
if (REASON_FOR_VISIT.substr(indexCurrent, 1) != '<') {
if (REASON_FOR_VISIT.substring(indexCurrent, REASON_FOR_VISIT.indexOf('<', indexCurrent
)).search(/[^\s]/g) != -1) {
if (strRFVConcat != '') {
strRFVConcat += '; '
}
strRFVConcat += REASON_FOR_VISIT.substring(indexCurrent, REASON_FOR_VISIT.indexOf('<',
indexCurrent));
}
}
indexCurrent = REASON_FOR_VISIT.indexOf('>', indexCurrent)+1;
}
fnDBColumnInsert('REASON_FOR_VISIT', strRFVConcat.substring(0, 1500));
}

fnDBColumnInsert('ENCOUNTER_DETAILS_DESCRIPTION', next.getField(encompassingEncounter + '/code/
@displayName'));
fnDBColumnInsert('DISCHARGE_DISPOSITION_CODE', fnTruncateToMax(sectionEncounters + '/entry/enc
ounter[templateId/@root="2.16.840.1.113883.10.20.22.4.49"]/sdct:dischargeDispositionCode' +
' || '
+ sectionEncounters + '/entry/encounter[templateId/@root="2.16.840.1.113883.10
.20.22.4.49"]/dischargeDispositionCode', 250));
fnDBColumnInsert('DISCHARGE_DISPOSITION_TEXT', fnTruncateToMax(sectionEncounters + '/entry/enc
ounter[templateId/@root="2.16.840.1.113883.10.20.22.4.49"]/sdct:dischargeDispositionCode/@dis
playName' + ' || '
+ sectionEncounters + '/entry/encounter[templateId/@root="2.16.840.1.113883.10
.20.22.4.49"]/dischargeDispositionCode/@displayName', 500));
if (next.getField(sectionPlanOfTreatment + '/entry/observation[templateId[1]/@root="2.16.840.1
.113883.10.20.22.4.44"]') != null) {
fnDBColumnInsert('PLANNED_ORDERS_FL', '1');
}
else { fnDBColumnInsert('PLANNED_ORDERS_FL', '0'); }

dbInsertsXML.appendChild(tempEle);

//
=====
// _____ SET eICR_DOCUMENT_PARSED XML NODES (end
)

```

Javascript

```
//
=====

//
=====
// _____ SET eICR_DOCUMENT_PATIENT XML NODES (start
)
//
=====
// Create XML nodes for key fields from the eICR document to be written to the eICR_DOCUMENT
_PATIENT table.

tempEle = new XML(<eICR_DOCUMENT_PATIENT></eICR_DOCUMENT_PATIENT>);

var patientRole = '/ClinicalDocument/recordTarget/patientRole';

//MRN
// Try the id with assigningAuthorityName="MRN" first, if not found, use the last id that is
not a SSN.
var MRN = '';

if (next.getField(patientRole + '/id[@assigningAuthorityName="MRN"]/@extension') != null) {
    MRN = next.getField(patientRole + '/id[@assigningAuthorityName="MRN"]/@extension').substring
(0, 150);
}
else {
    var cntMRN = next.getRepeatCount(patientRole + '/id');
    while (cntMRN > 0) {
        if ((next.getField(patientRole + '/id['+cntMRN+']/@root') == '2.16.840.1.113883.4.1') ||
(next.getField(patientRole + '/id['+cntMRN+']/@assigningAuthorityName') == 'Social Security
Administration')) {
            cntMRN--;
        }
        else {
            MRN = next.getField(patientRole + '/id['+cntMRN+']/@extension').substring(0, 150);
            cntMRN = 0;
        }
    }
}
fnDBColumnInsert('MRN', MRN);

fnDBColumnInsert('PATIENT_FIRST_NAME', fnTruncateToMax(patientRole + '/patient/name[@use="L"]/
given[1]+'||'
    +patientRole + '/patient/name[validTime/@high="nullFlavor"]/given[1]+'||'
    +patientRole + '/patient/name[1]/given[1]', 250));
fnDBColumnInsert('PATIENT_MIDDLE_NAME', fnTruncateToMax(patientRole + '/patient/name[@use="L"]/
given[2]+'||'
    +patientRole + '/patient/name[validTime/@high="nullFlavor"]/given[2]+'||'
    +patientRole + '/patient/name[1]/given[2]', 250));
fnDBColumnInsert('PATIENT_LAST_NAME', fnTruncateToMax(patientRole + '/patient/name[@use="L"]/
family+'||'
    +patientRole + '/patient/name[validTime/@high="nullFlavor"]/family+'||'
    +patientRole + '/patient/name[1]/family', 250));
fnDBColumnInsert('PATIENT_SUFFIX', fnTruncateToMax(patientRole + '/patient/name[@use="L"]/
suffix+'||'

```

Javascript

```

+patientRole + '/patient/name[validTime/@high="nullFlavor"]/suffix'+ '||'
+patientRole + '/patient/name[1]/suffix', 50));

//Address element
// Try most recent use="HP" first, if not found try most recent use="H", if not found get
address with most recent useablePeriod low, else use first address.
if (next.getField(patientRole + '/addr[@use="HP"]') != null) {
    //Check if there are multiple addresses with use="HP". If so, check the most recent useable
Period/low/@value.
    fnGetMostRecentAddress(patientRole + '/addr[@use="HP"]');
}
else if (next.getField(patientRole + '/addr[@use="H"]') != null) {
    fnGetMostRecentAddress(patientRole + '/addr[@use="H"]');
}
else {
    //Extract the address with the most recent useablePeriod/low/@value.
    fnGetMostRecentAddress(patientRole + '/addr');
}

fnCreateTelecomNode(patientRole + '/telecom[@use="MC"]/@value'+ '||'
    + patientRole + '/telecom[@use="HP"]/@value'+ '||'
    + patientRole + '/telecom[@use="H"]/@value'+ '||'
    + patientRole + '/telecom/@value', 'PATIENT_EMAIL', 250, 'PATIENT_PHONE', 250);
fnDBColumnInsert('PATIENT_BIRTH_SEX', fnTruncateToMax(sectionSocialHistory + '/entry/observa
tion[code/@code="76689-9"]/value/@code', 25));
fnDBColumnInsert('PATIENT_CURRENT_SEX', fnTruncateToMax(patientRole + '/patient/administrative
GenderCode/@code', 25));
fnDBColumnInsert('PATIENT_LANGUAGE', fnTruncateToMax(patientRole + '/patient/languageCommunica
tion/languageCode/@code', 500));
fnDBColumnInsert('PATIENT_DOB', fnTruncateToMax(patientRole + '/patient/birthTime/@value',
100));
fnDBColumnInsert('PATIENT_DOB_CONVERTED', fnConvertDOBField(next.getField(patientRole + '/pat
ient/birthTime/@value')));
fnDBColumnInsert('PATIENT_RACE', fnTruncateToMax(patientRole + '/patient/raceCode/@code
'+ '||'+patientRole + '/patient/sdtc:race/@code', 250));
fnDBColumnInsert('PATIENT_RACE_CODE_SYSTEM', fnTruncateToMax(patientRole + '/patient/raceCode/
@codeSystem'+ '||'+patientRole + '/patient/sdtc:race/@codeSystem', 250));
fnDBColumnInsert('PATIENT_RACE_NAME', fnTruncateToMax(patientRole + '/patient/raceCode/@display
Name'+ '||'+patientRole + '/patient/sdtc:race/@displayName', 500));
fnDBColumnInsert('PATIENT_ETHNICITY', fnTruncateToMax(patientRole + '/patient/ethnicGroupCode/
@code'+ '||'+patientRole + '/patient/sdtc:ethnicGroupCode/@code', 250));
fnDBColumnInsert('PATIENT_ETHNICITY_CODE_SYSTEM', fnTruncateToMax(patientRole + '/patient/
ethnicGroupCode/@codeSystem'+ '||'+patientRole + '/patient/sdtc:ethnicGroupCode/@codeSystem',
250));
fnDBColumnInsert('PATIENT_ETHNICITY_NAME', fnTruncateToMax(patientRole + '/patient/ethnicGroup
Code/@displayName'+ '||'+patientRole + '/patient/sdtc:ethnicGroupCode/@displayName', 500));

if (next.getField(patientRole + '/id[@root="2.16.840.1.113883.4.1"]/@extension') != null) {
    fnDBColumnInsert('PATIENT_SSN_FL', '1');
}
else { fnDBColumnInsert('PATIENT_SSN_FL', '0'); }

fnDBColumnInsert('PATIENT_MARITAL_STATUS', fnTruncateToMax(patientRole + '/patient/marital
StatusCode/@code', 250));
fnDBColumnInsert('PATIENT_MARITAL_STATUS_CODE_SYSTEM', fnTruncateToMax(patientRole + '/patient/
maritalStatusCode/@codeSystem', 250));
fnDBColumnInsert('PATIENT_MARITAL_STATUS_NAME', fnTruncateToMax(patientRole + '/patient/marital
StatusCode/@displayName', 500));

```

Javascript

```

fnDBColumnInsert('PATIENT_DECEASED_INDICATOR', fnTruncateToMax(patientRole + '/patient/sdtc:deceasedInd/@value', 50));
fnDBColumnInsert('PATIENT_DECEASED_DATE', fnTruncateToMax(patientRole + '/patient/sdtc:deceasedTime/@value', 100));
fnDBColumnInsert('PATIENT_DECEASED_DATE_CONVERTED', fnConvertDateTimeField(next.getField(patientRole + '/patient/sdtc:deceasedTime/@value')));
if (next.getField(patientRole + '/patient/administrativeGenderCode/@code') != 'M') {
    if (fnCurrentlyPregnant() == 'YES') { fnDBColumnInsert('PATIENT_PREGNANCY_STATUS', 'Currently Pregnant'); }
    fnDBColumnInsert('INFERRED_PREGNANCY_STATUS', fnTruncateToMax('/ClinicalDocument/component/structuredBody/component/section/entry/observation[templateId/@root="2.16.840.1.113883.10.20.15.3.8"]/value/@displayName' + '||'
        + '/ClinicalDocument/component/structuredBody/component/section/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.293"]/value/@displayName', 150));
}

if (next.getField(sectionSocialHistory + '/entry/act[templateId/@root="2.16.840.1.113883.10.20.15.2.3.1"]') != null) {
    fnDBColumnInsert('TRAVEL_HISTORY_LIKELY_FL', '1');
}
else { fnDBColumnInsert('TRAVEL_HISTORY_LIKELY_FL', '0'); }

fnDBColumnInsert('PATIENT_SMOKING_CODE', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/@code="72166-2"]/value/@code'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/translation/@code="72166-2"]/value/@code', 150));
fnDBColumnInsert('PATIENT_SMOKING_CODE_SYSTEM', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/@code="72166-2"]/value/@codeSystemName'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/translation/@code="72166-2"]/value/@codeSystemName', 250));
fnDBColumnInsert('PATIENT_SMOKING_CODE_DISPLAY', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/@code="72166-2"]/value/@displayName'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.78" and code/translation/@code="72166-2"]/value/@displayName', 1000));
fnDBColumnInsert('PATIENT_ALCOHOL_CODE', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code = "160573003"]/value/@code'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="11331-6"]/value/@code' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="11331-6"]/value/@code', 150));
fnDBColumnInsert('PATIENT_ALCOHOL_CODE_SYSTEM', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code = "160573003"]'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="11331-6"]/value/@codeSystemName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="11331-6"]/value/@codeSystemName', 250));
fnDBColumnInsert('PATIENT_ALCOHOL_CODE_DISPLAY', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code = "160573003"]'
    + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="11331-6"]/value/@displayName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="11331-6"]/value/@displayName', 1000));

```

Javascript

```

fnDBColumnInsert('PATIENT_TOBACCO_CODE', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="88031-0"]/value/@code' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="88031-0"]/value/@code' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/@code="11367-0"]/value/@code' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/translation/@code="11367-0"]/value/@code', 150));
fnDBColumnInsert('PATIENT_TOBACCO_CODE_SYSTEM', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="88031-0"]/value/@codeSystemName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="88031-0"]/value/@codeSystemName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/@code="11367-0"]/value/@codeSystemName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/translation/@code="11367-0"]/value/@codeSystemName', 250));
fnDBColumnInsert('PATIENT_TOBACCO_CODE_DISPLAY', fnTruncateToMax(sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/@code="88031-0"]/value/@displayName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.38" and code/translation/@code="88031-0"]/value/@displayName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/@code="11367-0"]/value/@displayName' + '||'
    + sectionSocialHistory + '/entry/observation[templateId[1]/@root="2.16.840.1.113883.10.20.22.4.85" and code/translation/@code="11367-0"]/value/@displayName', 1000));

var patientHPIExists = 0;
if (next.getField(sectionHistoryOfPresentIllness) != null) {
    var sectionHistoryOfPresentIllness_text = next.getField(sectionHistoryOfPresentIllness + '/text');
    var sectionHistoryOfPresentIllness_attrID = next.getField(sectionHistoryOfPresentIllness + '//@ID');

    if (sectionHistoryOfPresentIllness_text != null) {
        sectionHistoryOfPresentIllness_text = sectionHistoryOfPresentIllness_text.toString().toLowerCase();
        if (sectionHistoryOfPresentIllness_text.indexOf('nof') == -1 && sectionHistoryOfPresentIllness_text.indexOf('not on file') == -1 && sectionHistoryOfPresentIllness_text.indexOf('no history of present illness') == -1) {
            patientHPIExists = 1;
        }
        if (next.getField(patientRole + '/patient/administrativeGenderCode/@code') != 'M') {
            if (sectionHistoryOfPresentIllness_text.indexOf('pregnan') != -1) {
                fnDBColumnInsert('PATIENT_PREGNANCY_NOTES_FL', '1');
            }
            else { fnDBColumnInsert('PATIENT_PREGNANCY_NOTES_FL', '0'); }
        }
        else { fnDBColumnInsert('PATIENT_PREGNANCY_NOTES_FL', '0'); }
    }

    if (sectionHistoryOfPresentIllness_attrID != null) {
        sectionHistoryOfPresentIllness_attrID = sectionHistoryOfPresentIllness_attrID.toString().toLowerCase();
        if (sectionHistoryOfPresentIllness_attrID.indexOf('nof') == -1 && sectionHistoryOfPresentIllness_attrID.indexOf('not on file') == -1 && sectionHistoryOfPresentIllness_attrID.indexOf('no history of present illness') == -1) {

```

Javascript

```

    patientHPIExists = 1;
  }
}
}
fnDBColumnInsert('PATIENT_HPI_FL', patientHPIExists);

if ((next.getField(sectionImmunizations + '/entry/substanceAdministration[templateId/@root="2.16.840.1.113883.10.20.22.4.52"]/@moodCode') != null)
  && (next.getField(sectionImmunizations + '/entry/substanceAdministration[templateId/@root="2.16.840.1.113883.10.20.22.4.52"]/@moodCode') == 'EVN'
    || next.getField(sectionImmunizations + '/entry/substanceAdministration[templateId/@root="2.16.840.1.113883.10.20.22.4.52"]/@moodCode') == 'INT')) {
  fnDBColumnInsert('IMMUNIZATION_HISTORY_FL', '1');
}
else { fnDBColumnInsert('IMMUNIZATION_HISTORY_FL', '0'); }

dbInsertsXML.appendChild(tempEle);

//
=====
// _____ SET eICR_DOCUMENT_PATIENT XML NODES (end
)
//
=====

//
=====
// _____ SET eICR_DOCUMENT_MEDICATION XML NODES (start
)
//
=====

// Create XML nodes for key fields from the eICR document to be written to the eICR_DOCUMENT
_MEDICATION table.

if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID_Pairs') !=
null) {
  var tempMED_GROUP = new XML(<eICR_DOCUMENT_MEDICATION></eICR_DOCUMENT_MEDICATION>);

  //Most of the time, there will only be one condition but this while loop takes care of mul
tiple conditions.
  var arrConditions = next.getProperty('conditionCode').split('^');

  for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++) {
    var relevantCondition = arrConditions[cntConditions];
    var conditionCodeID = fnConditionCodeID(cntConditions);

    if (conditionCodeID != null) {
      //Loop through the Medications Administered section in the document and extract values only
for the medications relevant to the triggered condition.
      if (next.getField(sectionMedications + '/entry') != null) {
        for (var i = 1; i <= next.getRepeatCount(sectionMedications + '/entry'); i++) {
          var medCode = '';
          if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/
manufacturedProduct/manufacturedMaterial/code/@code') != null) {

            medCode = next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/con
sumable/manufacturedProduct/manufacturedMaterial/code/@code');

```

Javascript

```

    }
    else if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/translation[@codeSystem = "2.16.840.1.113883.6.88"]/@code') != null) {
        medCode = next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/translation[@codeSystem = "2.16.840.1.113883.6.88"]/@code');
    }
    if (fnIsRelevantMedication(relevantCondition, medCode)) {
        tempEle = new XML(<row_eICR_DOCUMENT_MEDICATION></row_eICR_DOCUMENT_MEDICATION>);

        fnDBColumnInsert('REPORTABLE_CONDITION_ID', conditionCodeID);

        if ((next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/@displayName') == null) &&
            (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/originalText') == null)) {
            var mednameFromTable = qryConditionMedsAndLabs.getField('/message/MEDICATION_CODES[MEDICATION_CODE="' + medCode + '" and CONDITION_CODE="' + relevantCondition + '"]/MEDICATION_DESCRIPTION');
            if (mednameFromTable != null) {
                fnDBColumnInsert('MEDICATION_NAME', mednameFromTable.substring(0, 500));
            }
        }
        else {
            fnDBColumnInsert('MEDICATION_NAME', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/@displayName' + '||' +
                sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/originalText', 500));
        }

        if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/@code') != null) {
            fnDBColumnInsert('MEDICATION_CODE', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/@code', 500));
        }
        else {
            fnDBColumnInsert('MEDICATION_CODE', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/consumable/manufacturedProduct/manufacturedMaterial/code/translation[@codeSystem = "2.16.840.1.113883.6.88"]/@code', 500));
        }

        fnDBColumnInsert('MEDICATION_DOSE_VALUE', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/doseQuantity/@value', 250));
        fnDBColumnInsert('MEDICATION_DOSE_UNIT', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/doseQuantity/@unit', 150));
        fnDBColumnInsert('MEDICATION_START_TIME', fnTruncateToMax(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/low/@value' + '||' + sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/@value', 150));
        if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/low/@value') != null) {
            fnDBColumnInsert('MEDICATION_START_TIME_CONVERTED', fnConvertDateTimeField(next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/low/@value')));
        }
        else if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/@value') != null) {

```

Javascript

```

        fnDBColumnInsert('MEDICATION_START_TIME_CONVERTED', fnConvertDateTimeField(next.getField(
(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/@value')));
    }
    fnDBColumnInsert('MEDICATION_END_TIME', fnTruncateToMax(sectionMedications + '/entry['+i
+']/substanceAdministration/effectiveTime/high/@value', 150));
    fnDBColumnInsert('MEDICATION_END_TIME_CONVERTED', fnConvertDateTimeField(next.getField(
(sectionMedications + '/entry['+i+']/substanceAdministration/effectiveTime/high/@value')));
    fnDBColumnInsert('MEDICATION_STATUS_CODE', fnTruncateToMax(sectionMedications + '/entry
['+i+']/substanceAdministration/statusCode/@code', 150));

    if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/@moodCode')
== 'INT') {
        fnDBColumnInsert('MEDICATION_ORDERED_FL', '1');
    }
    else { fnDBColumnInsert('MEDICATION_ORDERED_FL', '0'); }

    if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == 'not-done' ||
        next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == 'entered-in-error' ||
        next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == 'stopped' ||
        next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == 'aborted' ||
        next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == '' ||
        next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == null) {
        fnDBColumnInsert('MEDICATION_ABORTED_FL', '1');
    }
    else { fnDBColumnInsert('MEDICATION_ABORTED_FL', '0'); }

    if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/statusCode/
@code') == 'completed') {
        fnDBColumnInsert('MEDICATION_COMPLETED_FL', '1');
    }
    else { fnDBColumnInsert('MEDICATION_COMPLETED_FL', '0'); }

    if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/@moodCode')
== 'EVN') {
        fnDBColumnInsert('MEDICATION_ROUTE_GIVEN', fnTruncateToMax(sectionMedications + '/entry
['+i+']/substanceAdministration/routeCode/@displayName', 250));
    }
    if (next.getField(sectionMedications + '/entry['+i+']/substanceAdministration/@moodCode')
== 'INT') {
        fnDBColumnInsert('MEDICATION_ROUTE_ORDERED', fnTruncateToMax(sectionMedications + '/'
entry['+i+']/substanceAdministration/routeCode/@displayName', 250));
    }

    tempMED_GROUP.appendChild(tempEle);
} // if (fnIsRelevantMedication(relevantCondition, medCode)) (END)
} // for (var i = 1; i < next.getRepeatCount(sectionMedications + '/entry'); i++) (END)
} // if (next.getField(sectionMedications + '/entry') != null) (END)
} // if (conditionCodeID != null) (END)
} // for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++)
(END)

```

Javascript

```

dbInsertsXML.appendChild(tempMED_GROUP);
} // if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID_Pairs') != null) (END)

//
=====
//_____ SET eICR_DOCUMENT_MEDICATION XML NODES (end
)_____
//
=====

//
=====
//_____ SET eICR_DOCUMENT_LAB_RESULT XML NODES (start
)_____
//
=====
// Create XML nodes for key fields from the eICR document to be written to the eICR_DOCUMENT_LAB_RESULT table.

if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID_Pairs') != null) {
    var tempLAB_GROUP = new XML(<eICR_DOCUMENT_LAB_RESULT></eICR_DOCUMENT_LAB_RESULT>);
    var tempLAB_ORDER_GROUP = new XML(<eICR_DOCUMENT_LAB_ORDER></eICR_DOCUMENT_LAB_ORDER>);

    //Most of the time, there will only be one condition but this while loop takes care of multiple conditions.
    var arrConditions = next.getProperty('conditionCode').split('^');

    for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++) {
        var relevantCondition = arrConditions[cntConditions];
        var conditionCodeID = fnConditionCodeID(cntConditions);

        if (conditionCodeID != null) {
            //----- RESULTS section -----
            //Loop through the RESULTS section in the document and extract values only for the lab tests relevant to the triggered condition.
            if (next.getField(sectionResults + '/entry') != null) {
                for (var i = 1; i <= next.getRepeatCount(sectionResults + '/entry'); i++) {
                    var testCode = '';
                    for (var j = 1; j < next.getRepeatCount(sectionResults + '/entry['+i+']/organizer/component'); j++) {
                        if (next.getField(sectionResults + '/entry['+i+']/organizer/component['+j+']/observation/code/@code') != null) {
                            testCode = next.getField(sectionResults + '/entry['+i+']/organizer/component['+j+']/observation/code/@code');
                        }
                        else if (next.getField(sectionResults + '/entry['+i+']/organizer/component['+j+']/observation/code/translation/@code') != null) {
                            testCode = next.getField(sectionResults + '/entry['+i+']/organizer/component['+j+']/observation/code/translation/@code');
                        }
                    }
                    if (fnIsRelevantLabTest(relevantCondition, testCode)) {
                        tempEle = new XML(<row_eICR_DOCUMENT_LAB_RESULT></row_eICR_DOCUMENT_LAB_RESULT>);

                        fnDBColumnInsert('REPORTABLE_CONDITION_ID', conditionCodeID);

```